Marks: /100

Assignment: Functions, Scoping and Abstraction

Type: Group Assignment

Needed Modules: 1, 2 and 3 ONLY

- Students should **ONLY use** programming constructs covered in modules 1, 2 and 3.
- **Submissions using programming concepts or formats that are not covered in modules 1, 2, and 3 will be penalized.**
- **Don't use while True or while Boolean flag particularly if there is a clear loop condition(s).**
- **Penalty with no limit could be applied for each violation.**
    - **Minimum penalty is 25% for each violation.**
    - **The whole submission could not be accepted.**
- **Late submission will not be accepted.**

Flight Booking System Scenario

Create a flight booking system that allows users to view available flights, book flights, and manage bookings. The program should allow the following features:

- Storing and loading flight information into/from a file.
- Viewing available flights.
- Viewing flights' bookings.
- Booking a flight.
- Canceling a flight booking.

Equipment and Materials

For this assignment, you will need:

- Python IDE

Instructions

- This assignment will need to be completed outside of class time. See the course schedule and Brightspace for exact due dates.

© 2026, Southern Alberta Institute of Technology

- There are many details involved in this assignment which could be challenging to get sorted out and working correctly. Therefore, it is strongly recommended to work on the assignment early and in Team.
- Working in a group, review the Scenario and the Program Requirements sections of this document.
- Equally divide the functions between the group members.
  - Each member is responsible for implementing and testing his/her assigned functions and ensuring it is working properly.
  - Members should review the work completed by other members.
  - Integrate all the functions and test the whole program to ensure that the program meets all the requirements.
  - Use constants when it is appropriate.
- The program should exactly implement the provided test runs (check test runs file)
  - Use the values provided in test runs to test your program.
  - Test the program in the same order shown in the test runs file.
- Check your program against the marking criteria for the submission.

Submission

- Only one copy is required per group, and any of the group members may submit the following files to Brightspace:
  - The code of the program that you implemented (.py file)
  - A copy of the output (.txt file)
  - A PDF/DOC peer assessment.

Program Requirements

- The application **MUST use functions** to well structure the program and ensure the code usability.
- The functions (i.e., name, parameters, body) MUST be exactly implemented as specified.
- Flights information is loaded from the flights file (e.g., flights.txt) when the program starts.

© 2026, Southern Alberta Institute of Technology

- If the user provides a file that does not exist, the program displays an error message and re-ask the user to input an existing file name.
- The program continues displaying the main menu until the user quits (i.e., option 5).
- When booking and cancelling a flight, the flights file should be updated.
- Use appropriate compound data types (e.g., list, tuple, dictionary) to store flights and bookings information.
- The program **MUST exactly create the provided sample test runs.**
- You must exactly implement the following functions:

Menu Option	Function	Description

1	**view_flights( )**	• Display all available flights in a formatted table.
		  • Loop through each flight in the flights list
		  • Format and display the details of each flight.
		  ○ The flight details should include flight number, source, destination, available seats, and price.
		• **Parameter(s):** list of flights
		• The function does not return any value including None.

2	**book_flight( )**	• Allow a passenger to book seats on an available flight
		• It performs the following steps:
		  • Prompt the user for flight number
		  • Search the flights list for a matching flight
		  • If flight is found:
		  ○ Prompt user for number of seats to book
		  ○ Check if there are enough available seats (i.e., requested seats ≤ available seats)
		  ○ If there are enough available seats
		  ▪ Update the flight's available seats.
		  ▪ Call save_flights() to update the flight file.
		  ▪ Creates a new booking string in the format passenger name,flight no,seats booked.
		  ▪ Adds the booking to the bookings list.
		  ▪ Display a success message
		  ○ If there are not enough seats, display an error message
		  • If flight is not found, display "Flight not found"

		• **<u>Parameter(s):</u>** passenger name, file name, flights list, and bookings list. • The function does not return any value including None.

3	**view_bookings( )**	• Displays all bookings for the current passenger.
		• It performs the following steps:
		  • **Iterate** through each booking in the bookings list.
		  • **Process** each booking string to extract booking information.
		  • **Check** if the provided passenger has a booking.
		  • **If a booking is found for this passenger**:
		  ○ Prints the flight number and seats booked for that booking.
		  • If no matching bookings are found, prints "You have no bookings."
		• **<u>Parameter(s):</u>** passenger name, and bookings list.
		• The function does not return any value including None.

4	**cancel_booking( )**	• It cancels an existing flight booking by removing it from the bookings file and returning the reserved seats to flight's seats.
		• It performs the following main steps:
		  • **It takes input** for the flight number to cancel.
		  • **It searches** the bookings list for a booking that matches the passenger's name and flight number.
		  • **If the flight is found**:
		  ○ It retrieves the number of seats to cancel.
		  ○ It updates the flight's available seats in the flights list by adding back the canceled seats.
		  ○ It saves the updated flight data to the flights file (e.g. flights.txt).
		  ○ It removes the booking from the bookings list.
		  ○ It displays a success message.
		  • **If the flight is not found**: it displays an error message.
		• **<u>Parameter(s):</u>** passenger name, the flights' file name, flights list, and bookings list.
		• It does not return any value including None.

© 2026, Southern Alberta Institute of Technology

5	Exit program	• Exits the program with a goodbye message.
Additional Required Functions		
	load_flights( )	• It reads flights data from the flight file (i.e., file_name) ○ The flight data is a comma separated values and includes flight number, source, destination, seats available, and price. • It stores the flights data in a list (i.e., flights list) and returns it. • **Parameter(s):** flights' file name. • It returns the flights list.
	save_flights( )	• It saves the flights list to the flights file. • Before saving the flight data, it must be formatted as comma separated values. • **Parameter(s):** flights' file name and flights list. • It does not return any value including None.
	main_menu( )	• It displays the main menu for the flight booking system. • It asks the user to select a menu option and return it. • **Parameter(s):** No parameters. • It returns the user selection.

	main( )	• It is the main entry point and control function for the flight booking system.
		• It displays a welcome banner.
		• It prompts the user for a flight data filename (e.g., flights.txt) and checks if the file exists.
		  ○ If the file is not found, it displays an error message and prompts again.
		  ○ If the file is found, it loads the flights data from the file and stores the flights data in a list.
		• It asks for the passenger's name.
		• It continuously displays the main menu until the user chooses to exit (option 5).
		• It calls corresponding functions based on user selection
		• It displays an invalid input message if the user selects an invalid option

© 2026, Southern Alberta Institute of Technology

- You could create additional functions to reduce the code redundancy.
- Ensure appropriate documentation of all functions.

Test Plan

- Check test_runs.pdf for sample/test runs.

Peer assessment

- The peer assessment should be completed accurately and honestly
  - Deduction will be applied if it is not
- Each member should assess the contribution/participation of all other group members.
- Each member should assign a mark out of 10 to other members.
- Accurately describe the task completed by each member
- Please use the following table to complete the peer assessment. Marks/tasks are just samples.

	Member 1 Name	Member 2 Name	Member 3 Name	Completed Tasks
Member 1 Name	N/A	9	10	• Task 1 description • Task 2 description
Member 2 Name	8	N/A	9	• Task 3 description • Task 4 description
Member 3 Name	9	10	N/A	• Task 5 description • Task 6 description
Average	8.5	9.5	9.5	

A student can receive a deduction on their individual grade for the assignment if they do not meet the expected contribution/participation based on the feedback of other group members.

Marking Criteria

- Check Brightspace for marking rubrics.

© 2026, Southern Alberta Institute of Technology